

Agent is anything that can be viewed as: perceiving its env through sensors (eyes, ears) and acting upon that through actuators (legs, hand, mouth)

AI is considered with **consequentialism**. **Utilitarianism**: evaluated by scoring events and outcomes, and adding them for comparison

Task env: a problem specification for which the agent is a solution. **PEAS**: to specify a task environment, Specifying automated taxi driver

Perf measure: safe, fast, legal, comfortable, maximize profits. **Environment**: roads, other cars/traffic, roads, people, customers. **Actuators**: steering, pedal, brake, horn, signal. **Sensors**: cameras, speedometer, GPS. **Episodic**: does not affect what you face later. next episode doesn't depend on action taken in prev episode. **Sequential**: sequence of actions that affect what you face later.

Stochastic: random. Same input can lead to diff output. **Non-det**: not tech random bit multiple outcomes are possible due to external events.

Deterministic: No randomness involved. Returns the same output for the same input.

Static vs dynamic: if the env changes while the agent is performing. **Full Observable**: At each step agent's percepts (sensors) give it a complete view of current to make decisions. **Discrete**: grid cells, {up, down, left}. **Continuous**: test scores, blood test, angles, velocity

Reflex agent: selects action based on current percept and ignores hist. **Problem solving**: plans ahead and considers a sequence of actions that form path to goal.

PPT 2 Transition model: what action/results in. action suck - removes dirt. State space: set of all states reachable. Frontier states that are available for expanding (apply legal actions too)

Informed Search: Some non-goal nodes can be ranked above others. **Uninformed** search: all non-goal nodes in frontier look equally good.

Dimensions for evaluation: **completeness** - always find the solution? **Optimality** - does the search guarantee that the sol it returns is the best.

Time & space complexity. And their variables - **b**: max branch factor of search tree (the most # children a node can have). **d**: depth of shallowest GOAL node. **M** - # of steps/actions you might have to take from the start before you hit the deepest reachable state in search space

Breadth First Search: frontier is a FIFO queue **DFS**: frontier is LIFO. Comparison: Use DFS if space is restricted, many possible solutions with long paths and wrong paths are usually terminated quickly. Use BFS: possible infinite paths, some solutions have paths, can quickly discard unlikely paths.

The way iterative deepening works: reconstructs the nodes at each level i. So number of nodes constructed: $3 * b$ nodes + $2 * b^2$ nodes + b^3 nodes (generated nodes of depth 3 once).

Criterion	BFS	DFS	Depth Limited	Iterative deepening
Complete	Yes if b is finite	No (because even when b is finite, you can have infinite depth)	No (if $i < d$)	Yes (goal is always at depth d, the depth of the shallowest goal node)
Time	b^d	b^m	b^i	b^d
space	b^d	$b * m$ (good)	$b * i$ (good)	$b * d$ (good)
Optimal	Yes if cost=1 per step	No (can go deeply into right child with 100 steps, when sol is in left child)	No when $i > d$ Because it behaves like DFS.	Yes if step cost is 1

PPT 3 (INFORMED SEARCH)

Uniform Cost Search: frontier is a *priority queue ordered by g(n)*: depth of current node in search tree. Difference from BFS: test if node is a goal state when it is selected for expansion, not when its added to frontier. Updates nodes on frontier if a better to same state is found. Why? Bc we want to find path with lowest cost, adding to the frontier helps us keep track of this. **Best first search**: based on evaluation func ($f(n)$ that includes estimate to goal) which frontier is sorted by. 1) **Greedy best first search**: $f(n) = h(n)$. Expands nodes that are estimated to be closest to goal.

Optimal? No, can miss shorter paths. Complete? No, can get stuck in loops. 2) **A* Search**: $f(n) = g(n) + h(n)$, if $h(n)$ is admissible then tree search optimal. It expands nodes in increasing f value,

Admissibility: optimistic heuristics slow down bad plans look less promising (bigger f) but can't lie by making a path look too expensive vs reality

Inadmissible: pessimistic heuristics break optimality by pushing good plans too far back on frontier and so they may never be expanded

Heuristic is admissible if: $0 \leq h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost to goal.

Manhattan distance: $\text{abs}(a.x - b.x) + \text{abs}(a.y - b.y)$. **Is Manhattan admissible? The 8-level question is admissible with it because you will always be optimistic about the cost. With the assignment problem it was not because it will OVERESTIMATE the heuristic.**

Consistent heuristics: subset of admissible heuristics obeying triangle inequality. $a + b > c$, $b + c > a$, $a + c > b$

Adversarial Search: **Initial state**, **successor function** (list of (move, state) pairs specifying legal moves). **Terminal test**. **Utility function**: give numerical value of terminal states (win, lose, draw). PLY: 2 ply: MAX moves then MIN moves. **ALPHA-BETA PRUNING**: keep track of two

additional values: alpha: Max's current LOWER bound on Max's outcome (Max will never allow a move that could lead to a worse score for MAX then alpha). Beta: Min will never let for a better score for MAX then beta as it has already something this bad for MAX. Stop evaluating branch when; **1) Max node: $v \geq \beta$** is backed up (MIN wont ever select) **2) Min Node: $v \leq \alpha$** because MAX will never select min node.

Expectimax Search: compute avg score under optimal play. Chance nodes have uncertain outcomes, we calculate the expected utility (weighted avg of children) **CSP:** sol to a CSP is complete, constant assignment. **Idea 1:** CSP as search problem, sol always found at depth n for n variables (DFS used). Branching factor at root: $n * d$ and next level $(n-1) * d$. N is the number of variables in domain. Tree has $n!d^n$ leaves. Var assignments are commutative (order you assign doesn't matter) but assignments are fixed so use tree. **Backtracking search in uninformed algo** for CSP (DFS chooses one variable at a time until no valid assignments are left and backtracks.) **Idea 2: improving backtracking efficiency - heuristics:** **heuristic 1:** (most constrained var) choose variable fewest legal values (MRV), **heuristic 2:** (most constraining var) Choose var with most constraints on remaining vars, **heuristic 3:** (least constraining var) given a var, choose the least constraining var **heuristic 4:** forward checking: keep track of remaining legal values for unassigned variables. Terminate when unassigned var has no remaining legal values. Forward check doesn't provide early detection for all failures. Use **constraint propagation**. **Idea 3: Inference in CSP:** 1. Search 2. Constraint propagation (inference): reduce possible values for variable if the val violates local consistency. Arc consistency: satisfies binary constraints (node consistency is for unary constraints but trivial). Before anything is assigned arc consistency can run and detects failures early. $X \rightarrow Y$ is consistent iff for every val of X there is some allowed y . **Goal:** Make every node arc consistent (both ways) with a neighbour. Given, d values, checking arc consistency takes $O(d^2)$ worst case.

Inefficiencies: 1. Only examine X_i when at least one neighbour has lost its value. 2. If X_i loses a value bc of inconsistencies with y_j , you don't have to evaluate $y_j \rightarrow x_i$ immediately 3. Removing value of X can only make Y arc-inconsistent with respect to X itself only check (y, x) is arc consistent. AC-3: Worst case: $O(n^2d^3)$ bc all nodes can be connected to each other. If there are d vals each arc is checked

d^2 time and put on the queue d times. **OTHER TECHNIQUES:** 1) **Chronological backtracking** when a branch fails. 2) **Backjumping:** create conflict sets using forward checking. When we hit an error, jump back to reassign one of the conflicting vars. 3) **SIMPLE CSP.** Can make top sort and march down the graph. 4) **Cutset conditioning:** cut cycle out of graph by making the problematic node a fixed value. 5) **Local Search:** Hill-like climbing search for nearby solutions. To apply CSPs: 1) allow states with unsatisfied constraints 2) operators assign var values. Value selection by Min-conflict heuristic. 1) assign value with variable at random 2) loop and pick var that is currently in conflict often random. Change the value until the min number of conflicts exists. Repeat until there are 0 conflicts or u hit step limit and reset.

The Waltz/Mackworth Constraint Propagation Algorithm

1. Assign every node in the constraint graph a set of all possible values
2. Repeat until there is no change in the set of values associated with any node:
 3. For each node i :
 4. For each neighboring node j in the picture:
 5. Remove any value from i which is not arc consistent with j .